.UBAfiuba
FACULTAD DE INGENIERÍA

Sistema de telemetría y detección de eventos para motocicletas

Ing. Tomás Enrique Botalla

Carrera de Especialización en Sistemas Embebidos

Director: Esp. Ing. Germán Eloy Cardozo (UNLaM)

Jurados:

Por definir
Por definir
Por definir

Ciudad Autónoma de Buenos Aires, abril de 2027

Resumen

En la presente memoria se describe el diseño e implementación de un sistema embebido de telemetría y detección de eventos para motocicletas. El trabajo integra adquisición de variables cinemáticas y geográficas, almacenamiento en tarjeta de memoria removible y comunicación con una aplicación móvil en el teléfono.

Para su implementación fueron clave conocimientos relacionados con programación de microcontroladores, protocolos de comunicación, pruebas de software, sistemas operativos de tiempo real y gestión de proyectos.

Índice general

Resumen I

1. Introducción general 1
1.1. Introducción a la telemetría embebida en motocicletas 1
1.2. Contexto y motivación 2
1.3. Estado del arte 2
1.4. Objetivos del trabajo 3
1.4.1. Objetivo general 3
1.4.2. Objetivos específicos 3
1.4.3. Alcance 4

2. Introducción específica 5
2.1. Plataforma de hardware 5
2.1.1. Recursos del microcontrolador empleados 5
2.1.2. Entorno de desarrollo y depuración 6
2.2. Componentes y módulos externos 6
2.3. Sistema operativo y middleware 8
2.3.1. Capas de software de terceros 8
2.3.2. Aplicación móvil y servicios en el teléfono 8
2.4. Protocolos y buses de comunicación 9

3. Diseño e implementación 11
3.1. Análisis del software 11

4. Ensayos y resultados 13
4.1. Pruebas funcionales del hardware 13

5. Conclusiones 15
5.1. Conclusiones generales 15
5.2. Próximos pasos 15

Bibliografía 17

Índice de figuras

1.1. Vista conceptual de un sistema de telemetría vehicular. 1

2.1. Interconexión lógica entre el microcontrolador y los periféricos del prototipo. 7
2.2. Capas de software en el microcontrolador y en el teléfono móvil. . 9

Índice de tablas

1.1. Telemetría comercial 3

2.1. Módulos del prototipo 7
2.2. Parámetros de buses 9

Capítulo 1

Introducción general

En este capítulo se introduce el dominio de la telemetría embebida aplicada a motocicletas. Se presentan el contexto, la motivación, el estado del arte y los objetivos del trabajo.

1.1. Introducción a la telemetría embebida en motocicletas

Existe una amplia variedad de sistemas embebidos en la actualidad. Se encuentran en lo cotidiano en electrodomésticos, equipos médicos, instrumentación industrial, sistemas de transporte, entre muchos otros. En todos los casos, un firmware dedicado ejecuta la lógica que define la funcionalidad del producto y condiciona su confiabilidad en el tiempo.

Un sistema embebido es un sistema computacional orientado a una función acotada, integrado en un producto mayor y sujeto a límites de tamaño, consumo, costo y tiempo de respuesta. A diferencia de un equipo de uso general, el hardware y el software se conciben de forma conjunta para adquirir señales, controlar un proceso o registrar información en tiempo acotado.

En el ámbito vehicular, los sistemas embebidos abordan distintos dominios de aplicación. Uno de ellos es la telemetría vehicular, que concentra la adquisición, sincronización, almacenamiento, y transmisión de magnitudes cinemáticas y físicas. En la figura 1.1 se representa de forma esquemática la cadena típica en la que intervienen el vehículo, un dispositivo embebido que adquiere y registra datos, y una etapa de visualización de la información para el usuario. En competición y en flotas profesionales, este tipo de soluciones están ampliamente extendidas. En el uso particular del conductor, la demanda se orienta hacia equipos compactos, confiables, y de costo moderado [1].

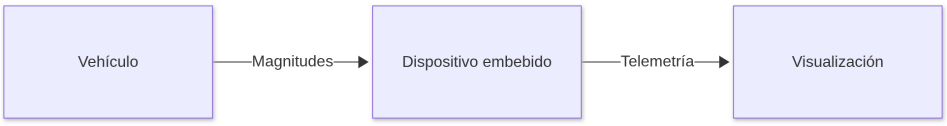


FIGURA 1.1. Vista conceptual de un sistema de telemetría vehicular.

1.2. Contexto y motivación

La conducción en motocicleta impone condiciones dinámicas exigentes. El usuario enfrenta cambios rápidos de velocidad, inclinación en curvas y mayor exposición a maniobras bruscas o a incidentes. Un dispositivo embebido montado en la motocicleta puede servir a la vez como instrumento de medición y como un respaldo de datos de conducción, con un rol análogo al de una caja negra en otros medios de transporte.

El proceso que se busca instrumentar es ese entorno de conducción. Se requiere registrar variables cinemáticas y geográficas, identificar maniobras bruscas o situaciones de riesgo y conservar un historial utilizable después del recorrido o ante un incidente. Muchos conductores recurren a aplicaciones en el teléfono móvil. Esas aplicaciones no están integradas al vehículo, dependen de sensores del teléfono y no garantizan registro autónomo en él. Los dispositivos profesionales ofrecen alta confiabilidad, pero exigen instalación, configuración y mantenimiento más complejos, además de costos elevados y ecosistemas cerrados poco accesibles para un uso particular.

Las restricciones de tamaño, consumo y costo se combinan con condiciones físicas exigentes en el vehículo, como vibración, variación térmica y espacio limitado para el montaje. Esas limitaciones obligan a una arquitectura optimizada. El microcontrolador debe coordinar periféricos en paralelo, sostener tasas de muestreo adecuadas y ejecutar lógica de análisis en tiempo acotado sin depender de servicios remotos permanentes. La plataforma debe reunir adquisición multivariable, persistencia local y enlace de corto alcance hacia una aplicación móvil en el teléfono.

La motivación del trabajo es cubrir esa brecha con un sistema embebido dedicado. Se prioriza la medición orientada al vehículo, el almacenamiento en tarjeta de memoria removible y el enlace inalámbrico sin conectividad celular en el dispositivo. El prototipo integra un microcontrolador STM32 (STMicroelectronics) sobre la placa de desarrollo NUCLEO-F446RE, sensado inercial, receptor de posicionamiento global, registro en tarjeta microSD y comunicación por Bluetooth de baja energía (BLE) con la aplicación móvil.

1.3. Estado del arte

Durante el relevamiento sobre telemetría embebida en motocicletas se constató que el mercado profesional está dominado por plataformas cerradas, probadas en competición y en desarrollo de producto [1]. Fabricantes como AiM Technologies, MoTeC y 2D Datarecording ofrecen dispositivos con unidad de medición inercial integrada, posicionamiento global, interfaces de bus de área de controlador (CAN), y software propietario de análisis [2, 3, 4, 5]. Esas soluciones alcanzan alta confiabilidad, pero suelen implicar costos elevados, formatos de datos poco abiertos y poca extensibilidad para algoritmos propios de análisis de conducción.

En la tabla 1.1 se resume un conjunto de las alternativas comerciales de nivel profesional relevadas.

En el ámbito académico, las publicaciones abordan registradores de eventos, reconstrucción de trayectorias, detección de colisiones, y plataformas embebidas

TABLA 1.1. Alternativas comerciales representativas de telemetría para vehículos y motocicletas.

Producto	Fabricante	Capacidades principales	Limitaciones
EVO5 [2]	AiM Technologies	Canales configurables, posicionamiento global, bus CAN, y software Race Studio	Arquitectura cerrada. Costo elevado en configuraciones completas
MXL2 [3]	AiM Technologies	Acelerómetro y giroscopio internos, posicionamiento global, entradas analógicas, y memoria interna	Mismo ecosistema propietario
C187 [4]	MoTeC	Display a color, registro de datos, posicionamiento global, y software MoTeC i2	Enfoque competición. Hardware y licencias de alto costo
Sticklogger V2/V3/V4 [5]	2D Datarecording	Registro en memoria USB 3.0, señales CAN, analógicas, y digitales, y configuración vía USB	Plataforma cerrada. Enfoque competición y entornos exigentes

de bajo costo. En Gelmini y col., se propuso un algoritmo de detección y clasificación de colisiones para vehículos de dos ruedas a partir de mediciones inerciales y de posicionamiento global [6]. En Smaiah y col., se reconstruyeron trayectorias de motocicleta con un sistema multisenorial de bajo costo [7]. En García Cuadra y col., se publicó una plataforma embebida abierta sobre microcontrolador STM32H7, con posicionamiento global de alta precisión, unidad de medición inercial, bus CAN con extensión CAN-FD, y enlace celular [1]. Ese aporte evidencia la viabilidad de arquitecturas abiertas frente a registradores cerrados de competición.

1.4. Objetivos del trabajo

Esta sección presenta los objetivos del trabajo y el alcance del prototipo desarrollado. Primero se define el objetivo general, seguido de los objetivos específicos. Finalmente, se establece el alcance del proyecto, detallando qué aspectos fueron implementados y cuáles quedaron excluidos.

1.4.1. Objetivo general

El objetivo general del trabajo fue diseñar e implementar un sistema embebido de telemetría y detección de eventos para motocicletas, capaz de adquirir variables cinemáticas y geográficas, registrar datos de forma local y comunicarse con una aplicación móvil en el teléfono.

1.4.2. Objetivos específicos

Los objetivos específicos fueron los siguientes:

1. Seleccionar e integrar una plataforma de hardware basada en la familia de microcontroladores STM32 y los módulos de sensado necesarios para posicionamiento global y medición inercial.
2. Desarrollar el firmware que permita adquirir, sincronizar y registrar muestras de sensores con tasas adecuadas para el análisis de la conducción.
3. Implementar almacenamiento persistente en una tarjeta de memoria removible, con formatos de registro que permitan un análisis posterior.
4. Establecer comunicación BLE con la aplicación móvil para la transferencia de telemetría.
5. Incorporar lógica para identificar maniobras de conducción relevantes a partir de las señales adquiridas, como base para puntuación y alertas.
6. Verificar el funcionamiento del sistema mediante ensayos sobre el hardware, el firmware, y la integración con la aplicación móvil.

1.4.3. Alcance

El alcance comprendió el diseño e implementación de un prototipo embarcado sobre la placa NUCLEO-F446RE, el firmware asociado y la aplicación móvil para el envío de telemetría por BLE. Se priorizó la adquisición multivariable, la persistencia local y la interfaz con el usuario móvil.

En el firmware se incluyó un sistema operativo de tiempo real para ejecutar de forma concurrente el muestreo de sensores, el registro en tarjeta de memoria removible y el enlace BLE. Esa organización buscó aislar las rutinas periódicas de las operaciones de entrada-salida de mayor latencia y mantener acotados los tiempos de respuesta del dispositivo.

La arquitectura de bloques del dispositivo y la interconexión de periféricos se presentan en el capítulo 2.

Quedaron fuera del alcance cualquier tipo de certificación u homologación del dispositivo, la conexión con la electrónica de la motocicleta mediante bus CAN u otros enlaces de diagnóstico, y el uso de servicios en la nube para almacenamiento remoto. Tampoco se buscó igualar el desempeño ni el costo de dispositivos comerciales de competición. El foco fue proveer una solución extensible y de costo moderado para conductores particulares.

Capítulo 2

Introducción específica

Este capítulo describe los componentes de hardware y software de terceros integrados en el prototipo de telemetría. En particular, se detallan el microcontrolador central, los módulos externos de sensado y comunicación, las capas de firmware base, y los protocolos de enlace físico. Por su parte, el diseño y la implementación a medida se abordan en el capítulo 3.

2.1. Plataforma de hardware

La plataforma de desarrollo seleccionada fue la placa NUCLEO-F446RE [8], que integra el microcontrolador STM32F446RET6 en encapsulado LQFP64 [9]. El núcleo es un procesador ARM Cortex-M4 con unidad de punto flotante y un conjunto de periféricos orientados a adquisición de datos, temporización y comunicación serie.

La STM32F446RET6 permite operar el núcleo Cortex-M4 hasta 180MHz, garantizando así la capacidad de cálculo necesaria para procesar con rapidez los datos provenientes de los distintos periféricos. Además, la placa integra un depurador ST-LINK con soporte para puerto COM virtual (UART), lo que permite registrar datos y ejecutar pruebas desde el entorno de desarrollo sin requerir hardware adicional.

La elección de esta plataforma se basó en la disponibilidad de múltiples periféricos (controladores UART, I2C y SPI) accesibles a través de los conectores Arduino y Morpho. Esto resultó fundamental para interconectar simultáneamente el receptor GPS, la unidad de medición inercial, el módulo Bluetooth, y la memoria extraíble. Finalmente, el amplio ecosistema de desarrollo provisto por STMicroelectronics facilitó la implementación del firmware, agilizando la inicialización del proyecto y la adopción de diferentes middlewares y de la capa de abstracción de hardware (HAL).

2.1.1. Recursos del microcontrolador empleados

En el integrado STM32F446RET6 se empleó la memoria flash interna para el código fuente, y la SRAM interna para pilas, colas circulares y buffers de los distintos canales. Esa organización permitió alojar el sistema de archivos FatFs [10] y las estructuras de telemetría sin recurrir a memorias externas de programa.

La asignación de los periféricos de comunicación se estableció en función del diseño de hardware del prototipo. En particular, se configuraron tres interfaces

UART: una para la recepción de tramas NMEA del módulo GPS, otra para el enlace con el módulo Bluetooth de baja energía, y una tercera destinada a la consola de depuración vía ST-LINK. Por su parte, la comunicación con la unidad inercial (LSM6DSO) se resolvió mediante el bus I2C1, mientras que el bus SPI2 se destinó al controlador de la tarjeta microSD para gestionar las transferencias en bloque requeridas por el middleware de archivos.

Con el propósito de optimizar la adquisición de datos, se habilitó el acceso directo a memoria (DMA, *Direct Memory Access*) exclusivamente en los canales de mayor tráfico: la recepción UART de tramas NMEA y las transferencias en bloque vía SPI hacia la memoria microSD. Esta configuración permitió direccionar los datos hacia buffers circulares de forma asíncrona, minimizando la intervención de la CPU ante ráfagas de comunicación y operaciones de escritura. En contraste, los enlaces I2C con la unidad inercial y UART de depuración operaron sin DMA, dada su menor demanda de ancho de banda. Así, los recursos del procesador se concentraron en el muestreo inercial y la orquestación del registro, gobernados por los temporizadores de propósito general.

Finalmente, la tolerancia a fallos del firmware se garantizó mediante la integración de un IWDG (*Independent Watchdog*). Una tarea de supervisión del RTOS actualizó periódicamente su contador. Ante cualquier bloqueo prolongado del planificador o de los objetos activos, el sistema embebido se reinició de forma automática y recuperó un estado operativo seguro.

2.1.2. Entorno de desarrollo y depuración

El proyecto de firmware se generó y mantuvo con STM32CubeMX y se compiló con la cadena GNU para ARM (arm-none-eabi-gcc) integrada en STM32CubeIDE [11]. La configuración de pines, relojes y instancias de periféricos se versionó en un archivo de descripción del microcontrolador, de modo de reconstruir el hardware lógico ante cambios de cableado. La depuración y el registro de eventos en etapas intermedias se realizaron por el puerto virtual del ST-LINK, sin exigir un analizador lógico externo para la validación básica de los buses.

2.2. Componentes y módulos externos

El prototipo cablea al microcontrolador un conjunto de módulos de adquisición, comunicación y almacenamiento: receptor GNSS [12], unidad inercial de seis ejes, transceptor Bluetooth de baja energía y tarjeta microSD. A continuación se presenta una síntesis tabular y, en los párrafos siguientes, el detalle de acoplamiento de cada módulo.

En la tabla 2.1 se resume el conjunto de módulos conectados al microcontrolador, la interfaz física empleada y notas de configuración relevantes para el firmware.

El receptor GNSS NEO-M8N [13] entrega sentencias en formato NMEA [14] que el firmware interpreta para obtener posición y tiempo. Opera a 3.3 V, se enlaza por UART cruzada al microcontrolador y mantiene una tasa de 9600 baudios compatible con la configuración por defecto del módulo.

La unidad LSM6DSO [15] concentra acelerómetro y giroscopio de tres ejes en un solo encapsulado. La dirección I2C seleccionada (0x6A) corresponde al nivel lógico bajo del pin de dirección del integrado. El módulo se montó sobre el bus

TABLA 2.1. Módulos externos del prototipo y su acoplamiento al microcontrolador.

Módulo	Función	Interfaz	Notas de uso
NEO-M8N	Posicionamiento GNSS	USART1 a 9600 baudios	Trazas en formato NMEA, recepción UART por DMA
LSM6DSO	Medición inercial (6 ejes)	I2C1 a 100 kHz	Dirección 7 bits 0x6A, acelerómetro y giroscopio integrados
HC-10 (BT05)	Enlace Bluetooth de baja energía	USART3 a 9600 baudios	Módulo serie transparente, perfil GATT en el teléfono
Tarjeta microSD	Registro local	SPI2	Sistema de archivos FAT, registro de telemetría en CSV

I2C1 expuesto en los conectores Arduino de la placa Nucleo, lo que simplificó el cableado de prototipo.

El módulo HC-10 (variante BT05 sobre CC2541) [16] expuso una interfaz serie transparente hacia el firmware. El enlace de baja energía quedó del lado del controlador Bluetooth. Desde el microcontrolador la interacción se limitó a enviar y recibir octetos por UART, de forma análoga a un cable serie corto.

La tarjeta microSD se acopló mediante un módulo adaptador SPI con alimentación a 5 V según el esquema del breakout utilizado. El registro en archivo CSV permitió conservar series temporales aun cuando el teléfono no estuviera emparejado, y facilitó la revisión posterior de trayectos en una computadora.

En la figura 2.1 se representa de forma esquemática la interconexión lógica entre la placa NUCLEO-F446RE, los buses configurados en el firmware y los periféricos del prototipo.

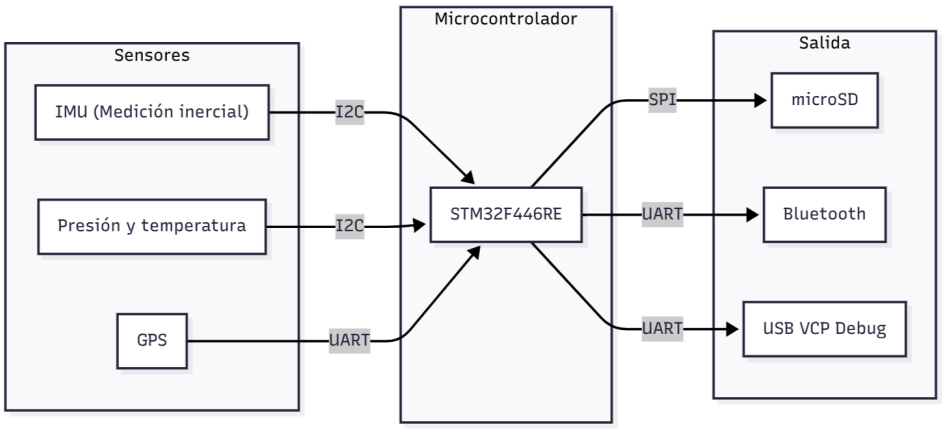


FIGURA 2.1. Interconexión lógica entre el microcontrolador y los periféricos del prototipo.

El diagrama condensa las relaciones de la tabla 2.1. El GNSS y el módulo Bluetooth emplearon UART dedicadas, la unidad inercial compartió el bus I2C1 con el integrado central, y la microSD se accedió por SPI2. El puerto serie del ST-LINK quedó disponible para depuración sin formar parte del lazo de telemetría en el vehículo.

2.3. Sistema operativo y middleware

El firmware del prototipo se ejecutó sobre un sistema operativo de tiempo real (RTOS, *Real-Time Operating System*). Se empleó FreeRTOS [17] integrado en el árbol de middleware de STM32Cube, con el planificador preemptivo por prioridades para asegurar plazos acotados en adquisición, registro y enlace inalámbrico. La arquitectura concurrente del firmware y el reparto en tareas y colas de mensajes se detallan en el capítulo 3.

2.3.1. Capas de software de terceros

Sobre el kernel RTOS se integraron las siguientes piezas de soporte:

- CMSIS y HAL de STM32, generados con STM32CubeMX a partir del paquete de firmware STM32Cube para la familia F4, para acceso a GPIO, temporizadores, DMA y periféricos serie.
- FreeRTOS, con APIs de tareas, colas, semáforos y temporizadores de software, como base del planificador y de la comunicación entre módulos de firmware.
- FatFs, integrado en el árbol de middleware del proyecto, para operar sobre la tarjeta microSD con partición FAT32.
- Herramientas de verificación en el host (Ceedling [18] y Unity [19]) para pruebas unitarias de componentes de firmware, sin formar parte del binario embarcado.

La capa HAL abstrajo los registros de hardware en rutinas de configuración y ocultó los detalles del fabricante detrás de tipos comunes para UART, I2C y SPI. CMSIS aportó definiciones del núcleo Cortex-M y vectores de interrupción coherentes con la documentación del fabricante. Las llamadas al sistema de archivos se ejecutaron desde las tareas de aplicación, con exclusión mutua frente a otras tareas que pudieran disputar el mismo medio.

2.3.2. Aplicación móvil y servicios en el teléfono

En el lado del teléfono móvil se empleó una aplicación web progresiva [20] desarrollada con React [21] y Vite [22]. La comunicación con el prototipo utilizó la API Web Bluetooth [23] del navegador, que expuso el perfil del módulo HC-10 hacia la capa de presentación (mapas y visualización de telemetría). Ese modelo evitó distribuir un binario nativo por medio de las tiendas de aplicaciones y permitió iterar la interfaz con las mismas tecnologías empleadas en el ecosistema web. Los mapas se apoyaron en la biblioteca Leaflet [24] de código abierto. La interpretación de las tramas recibidas por BLE residió en el cliente y no en el microcontrolador.

La figura 2.2 resume las relaciones entre las capas de software del microcontrolador y el cliente móvil. El detalle de tareas, colas de eventos, drivers y formatos de registro se describe en el capítulo 3.

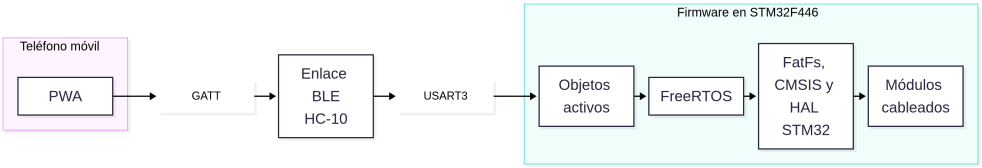


FIGURA 2.2. Capas de software en el microcontrolador y en el teléfono móvil.

2.4. Protocolos y buses de comunicación

Los enlaces físicos entre el STM32F446 y los módulos externos se implementaron con tres familias de interfaz habituales en adquisición de sensores y almacenamiento local. En la tabla 2.2 se resumen las instancias configuradas en el prototipo.

TABLA 2.2. Parámetros de los buses serie y paralelos configurados en el prototipo.

Bus	Periférico	Tasa / modo	Rol en el sistema
USART1	NEO-M8N	9600 baudios	Recepción de tramas NMEA por DMA a buffer circular
USART2	ST-LINK VCP	115200 baudios	Consola de depuración hacia el host
USART3	HC-10 (BLE)	9600 baudios	Enlace serie transparente con el teléfono
I2C1	LSM6DSO	100 kHz, maestro	Lectura periódica de registros de la IMU
SPI2	microSD	Modo maestro, DMA	Almacenamiento de ficheros CSV vía FatFs

En la comunicación UART asíncrona se configuraron tres instancias USART para el GPS (9600 baudios), la depuración por ST-LINK (115200 baudios) y el módulo Bluetooth (9600 baudios). El enlace con el receptor NEO-M8N se apoyó en sentencias NMEA para obtener coordenadas y tiempo de referencia. La recepción en USART1 se realizó por DMA hacia un buffer circular, de modo de reducir la carga de interrupciones ante ráfagas de tramas. Las sentencias se acumularon en ese buffer y se descartaron tramas incompletas antes de interpretarlas. La depuración y el módulo HC-10 operaron sin DMA, con una carga de tráfico inferior. En el módulo BLE el UART actuó como túnel serie hacia el controlador CC2541. No se implementó un protocolo propietario encima de los octetos en el firmware embarcado.

El bus I2C1 conectó la IMU LSM6DSO en modo maestro a 100 kHz. Los registros del sensor se accedieron mediante direccionamiento de 7 bits y lecturas de bloques para aceleración y velocidad angular.

El bus SPI2 enlazó la tarjeta microSD en modo maestro con una línea de selección de esclavo dedicada. Las transferencias en bloque hacia la tarjeta se apoyaron en DMA, alineadas con lo descrito en la sección 2.1. Las operaciones de lectura y escritura se tradujeron a llamadas al middleware FatFs sobre ficheros en formato

CSV. Se asumió el uso de una partición FAT32 y un tamaño de cluster compatible con tarjetas de uso general de hasta 32 GB.

A nivel de aplicación móvil, el intercambio con el usuario se realizó mediante el perfil GATT expuesto por el módulo (servicio y característica de datos serie), mientras que la persistencia local en el vehículo dependió del sistema de archivos FAT ya referido en el microcontrolador. Las capas descritas en este capítulo se integran en el diseño detallado del capítulo 3, donde se explicitan los módulos de firmware y los formatos de registro adoptados.

Capítulo 3

Diseño e implementación

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

3.1. Análisis del software

La idea de esta sección es resaltar los problemas encontrados, los criterios utilizados y la justificación de las decisiones que se hayan tomado.

Se puede agregar código o pseudocódigo dentro de un entorno lstlisting con el siguiente código:

```
\begin{lstlisting}[caption= "un epígrafe descriptivo"]
 las líneas de código irían aquí...
\end{lstlisting}
```

A modo de ejemplo, se muestra el fragmento de código 3.1:

```
#define MAX_SENSOR_NUMBER 3
#define MAX_ALARM_NUMBER  6
#define MAX_ACTUATOR_NUMBER 6

uint32_t sensorValue[MAX_SENSOR_NUMBER];
FunctionalState alarmControl[MAX_ALARM_NUMBER]; //ENABLE or DISABLE
state_t alarmState[MAX_ALARM_NUMBER];           //ON or OFF
state_t actuatorState[MAX_ACTUATOR_NUMBER];     //ON or OFF

void vControl() {

  initGlobalVariables();

  period = 500 ms;

  while(1) {

    ticks = xTaskGetTickCount();

    updateSensors();

    updateAlarms();

    controlActuators();

    vTaskDelayUntil(&ticks, period);
  }
```

```
28 }
```

CÓDIGO 3.1. Pseudocódigo del lazo principal de control.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se puedo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

[1] Andrei García Cuadra, Alberto Brunete González y Francisco Santos Olalla. «Design and Experimental Evaluation of an Open-Architecture Multi-Sensor Telemetry System for Real-Time Motorcycle Dynamics Acquisition». En: *Electronics* 15.12 (2026), pág. 2604. DOI: 10.3390/electronics15122604. URL: https://doi.org/10.3390/electronics15122604 (visitado 01-11-2026).

[2] *EVO5: Data Acquisition System Technical Specification*. Hoja de producto. AiM Technologies. 2023. URL: https://www.aim-sportline.com/en/out-of-production/evo5/technical-specifications-and-installation.htm (visitado 01-11-2026).

[3] *MXL2 Dash Logger: Technical Specifications*. Hoja de producto. AiM Technologies. 2021. URL: https://www.aim-sportline.com/en/out-of-production/mxl2/technical-specifications.htm (visitado 01-11-2026).

[4] *C187 Colour Display Logger: Technical Specification*. Hoja de producto. MoTeC. 2023. URL: https://www.motec.com.au/products/C187 (visitado 01-11-2026).

[5] *Professional Motorcycle Data Acquisition Systems*. Catálogo de producto. 2D Datarecording. 2023. URL: https://2d-datarecording.com/datalogger-overview/ (visitado 01-11-2026).

[6] Simone Gelmini et al. «A Novel Crash Detection Algorithm for Two-Wheeled Vehicles». En: *IEEE Transactions on Intelligent Vehicles* 6.1 (2021), págs. 88-99. DOI: 10.1109/TIV.2020.3028462. URL: https://doi.org/10.1109/TIV.2020.3028462 (visitado 01-11-2026).

[7] Sarra Smaiah et al. «A Practical Approach for High Precision Reconstruction of a Motorcycle Trajectory Using a Low-Cost Multi-Sensor System». En: *Sensors* 18.7 (2018), pág. 2282. DOI: 10.3390/s18072282. URL: https://doi.org/10.3390/s18072282 (visitado 01-11-2026).

[8] *NUCLEO-F446RE: STM32 Nucleo-64 development board with STM32F446RE MCU*. STMicroelectronics. 2024. URL: https://www.st.com/en/evaluation-tools/nucleo-f446re.html (visitado 01-11-2026).

[9] *STM32F446xC/E: advanced ARM-based 32-bit MCUs*. STMicroelectronics. 2023. URL: https://www.st.com/resource/en/datasheet/stm32f446re.pdf (visitado 01-11-2026).

[10] ChaN. *FatFs – Generic FAT File System Module*. 2019. URL: http://elm-chan.org/fsw/ff/00index_e.html (visitado 01-11-2026).

[11] *STM32CubeIDE: integrated development environment for STM32 products*. STMicroelectronics. 2024. URL: https://www.st.com/en/development-tools/stm32cubeide.html (visitado 09-11-2026).

[12] National Coordination Office for Space-Based PNT. *Global Navigation Satellite Systems (GNSS)*. 2024. URL: https://www.gps.gov/systems/gnss/ (visitado 09-11-2026).

[13] *NEO-M8: u-blox M8 concurrent GNSS modules data sheet (variant NEO-M8N)*. Data sheet, document UBX-15031086, revision R14. u-blox. 2025. URL: https:

//content.u-blox.com/sites/default/files/NEO-M8-FW3_DataSheet_UBX-15031086.pdf (visitado 09-11-2026).

[14] *NMEA 0183 Interface Standard*. Página oficial de referencia (versión 4.30); el documento normativo completo es de distribución restringida. National Marine Electronics Association, 2024. URL: https://www.nmea.org/nmea-0183.html (visitado 09-11-2026).

[15] *LSM6DSO: iNEMO inertial module with 6-axis accelerometer and gyroscope*. STMicroelectronics. 2020. URL: https://www.st.com/resource/en/datasheet/lsm6dso.pdf (visitado 01-11-2026).

[16] *CC2541: SimpleLink Bluetooth low energy and proprietary wireless MCU*. Integrado empleado en módulos serie HC-10/BT05 con enlace UART transparente y servicio GATT FFE0/FFE1. Texas Instruments. 2013. URL: https://www.ti.com/lit/ds/symlink/cc2541.pdf (visitado 09-11-2026).

[17] *FreeRTOS kernel documentation*. Amazon Web Services. 2024. URL: https://www.freertos.org/Documentation/RTOS_book.html (visitado 09-11-2026).

[18] ThrowTheSwitch.org. *Ceedling project*. 2024. URL: https://github.com/ThrowTheSwitch/Ceedling (visitado 09-11-2026).

[19] ThrowTheSwitch.org. *Unity: unit testing framework for C*. 2024. URL: https://github.com/ThrowTheSwitch/Unity (visitado 09-11-2026).

[20] MDN Web Docs. *Progressive web apps*. 2025. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps (visitado 09-11-2026).

[21] React. *React: the library for web and native user interfaces*. 2025. URL: https://react.dev/ (visitado 09-11-2026).

[22] Vite Contributors. *Vite: next generation frontend tooling*. 2025. URL: https://vite.dev/ (visitado 09-11-2026).

[23] Web Bluetooth Community Group. *Web Bluetooth*. Draft Community Group Report, hospedado bajo el programa de grupos de la W3C. 2026. URL: https://webbluetoothcg.github.io/web-bluetooth/ (visitado 09-11-2026).

[24] Leaflet Contributors. *Leaflet: an open-source JavaScript library for mobile-friendly interactive maps*. 2025. URL: https://leafletjs.com/ (visitado 09-11-2026).